1. (A) Results from distinct n-Grams on each review:

n-Grams	D1	D2	D3	D4
2-grams (char)	333	461	401	154
3-grams (char)	933	1714	1428	256
2-grams (word)	330	875	571	54

Table 1. Distinct n-Grams for each review. See Listing 2 for implementation details.

The results from Table 1 were obtained by creating a `CountVectorizer` object with the following parameters, then applying the `fit` and `get_feature_names_out` methods:

```
-- CountVectorizer(analyzer = 'char', ngram_range = (2,2))
-- CountVectorizer(analyzer = 'char', ngram_range = (3,3))
-- CountVectorizer(analyzer = 'word', ngram_range = (2,2))
```

(B) A subroutine for generalized set similarity is implemented below:

```
def generalized_set_similarity_measure(A, B, C, D, x, y, z, z_prime):
    a = len(A.intersection(B))
    b = len(A.union(B).union(C).union(D)) - len(A.union(B))
    c = len(A.symmetric_difference(B))
    d = x * a + y * b + z * c
    e = x * a + y * b + z_prime * c
    return d / e
```

Listing 1. Generalized set similarity measure.

Below are the Jaccard similarities corresponding to every pair:

n-Grams	JS(D1,D2)	JS(D1,D3)	JS(D1,D4)	JS(D2,D3)	JS(D2,D4)	JS(D3,D4)
2-grams (char)	0.59118	0.62031	0.40751	0.66409	0.30851	0.34709
3-grams (char)	0.32682	0.31021	0.17839	0.40080	0.11932	0.12567
2-grams (word)	0.01516	0.01923	0.00524	0.02263	0.00324	0.00160

Table 2. Jaccard similarity on every document pair. See Listing 2 for implementation details.

2. (A) Results of the approximation of Jaccard similarity base on `fast-min-hash` algorithm:

t	~JS(D1,D2)
20	0.85000
60	0.90000
150	0.92000
300	0.90000
600	0.89833

Table 3. Approximation of Jaccard similarity based on `fast-min-hash` algorithm. See Listing 3 for implementation details.

(B) Benchmark results:

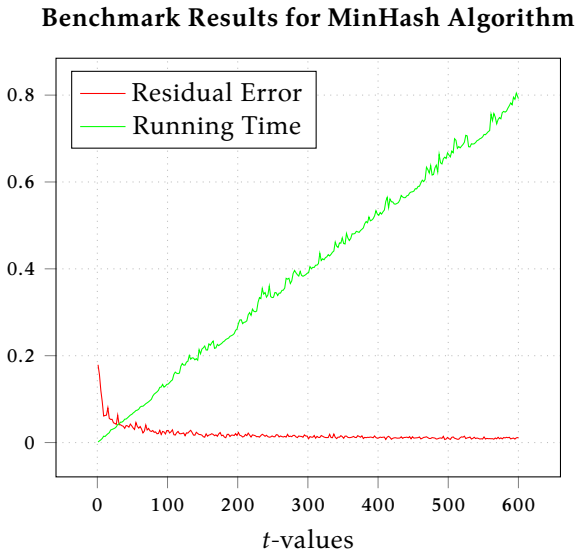

Figure 1. Benchmark results from `fast-min-hash`. Plotted values correspond to expectation on 100 trials. See Listing 3 for implementation details.

From Figure 1, we see that $t \approx 35$ produces the best approximation with the least runtime overhead. In general, a choice $t = 35$ would be appropriate if dealing with massively large text files. For the datasets provided, however, choosing $t \in [300, 400]$ would be ideal as values past $t > 400$ results in diminishing improvements in accuracy while continuing to introduce significant computational overhead to the running time.

Appendix

```python
import io
import argparse
import pandas as pd

from itertools import starmap, combinations
from sklearn.feature_extraction.text import CountVectorizer

def generalized_set_similarity_measure(
    A: set, B: set, C: set, D: set,
    x: int, y: int, z: int, z_prime: int
) -> float:
    a = len(A.intersection(B))
    b = len(A.union(B).union(C).union(D)) - len(A.union(B))
    c = len(A.symmetric_difference(B))
    d = x * a + y * b + z * c
    e = x * a + y * b + z_prime * c
    return d / e

def read_file(file: io.TextIOWrapper) -> list:
    data = pd.read_csv(file)
    corpus = list(data.text)
    corpus = corpus[:4]
    return corpus

def build_ngrams(corpus: str, ngram: int, token: str) -> set:
    vectorizer = CountVectorizer(analyzer=token, ngram_range=(ngram, ngram))
    _ = vectorizer.fit([corpus])
    features = set(vectorizer.get_feature_names_out())
    return features

def main() -> int:
    parser = argparse.ArgumentParser(prog='build-ngram', description='n-gram document builder')
    parser.add_argument('-i', '--input', type=argparse.FileType('r'), required=True, help='path to input file')
    parser.add_argument('-n', '--ngram', type=int, required=True, help='n-gram size')
    parser.add_argument('-t', '--token', type=str, required=True, choices=['char', 'word'], help='token to use')

    args = parser.parse_args()
    corpus = read_file(args.input)
    corpus_size = len(corpus)

    ngrams = list(
        starmap(
            build_ngrams,
            zip(corpus, corpus_size * [args.ngram], corpus_size * [args.token])
        )
    )

    for fst_ngram, snd_ngram in combinations(ngrams, 2):
        thd_ngram, fth_ngram = (ngram for ngram in ngrams if ngram not in (fst_ngram, snd_ngram))
        similarity = generalized_set_similarity_measure(
            fst_ngram, snd_ngram,
            thd_ngram, fth_ngram,
            1, 0, 0, 1
        )
        print(similarity)

    return 0

if __name__ == '__main__':
    raise SystemExit(main())
```

Listing 2. build-ngrams.py

```python
import io
import time
import random
import string
import argparse
import numpy as np

from itertools import starmap
from sklearn.feature_extraction.text import CountVectorizer

class Hash:
    def __init__(self, salt: str) -> None:
        self.salt = salt

    def hash_fn(self, item: str) -> int:
        return hash((self.salt, item))

def read_file(file: io.TextIOWrapper) -> list:
    corpus = file.read().splitlines()
    return corpus

def build_ngrams(corpus: list, ngram: int, token: str) -> set:
    vectorizer = CountVectorizer(analyzer=token, ngram_range=(ngram, ngram))
    _ = vectorizer.fit(corpus)
    features = set(vectorizer.get_feature_names_out())
    return features

def rand_salt(length: int) -> str:
    return ''.join(random.sample(string.ascii_letters + string.digits, length))

def min_hash_sign(s: set, hash_fns: list) -> list:
    sign_s = []
    for hash_fn in hash_fns:
        min_hash = min(hash_fn(item) for item in s)
        sign_s.append(min_hash)
    return sign_s

def jaccard_similarity(a: set, b: set) -> float:
    return len(a.intersection(b)) / len(a.union(b))

def minhash_similarity(a: set, b: set, t: int) -> float:
    hashes = [Hash(rand_salt(9)).hash_fn for _ in range(t)]
    sign_a = min_hash_sign(a, hashes)
    sign_b = min_hash_sign(b, hashes)
    return sum(1 for a, b in zip(sign_a, sign_b) if a == b) / t

def run_similarity(ngrams: list) -> None:
    for t in [20, 60, 150, 300, 600]:
        ms = minhash_similarity(ngrams[0], ngrams[1], t)
        js = jaccard_similarity(ngrams[0], ngrams[1])
        print(f'{t=:03}, {ms=:.3f}, {js=:.3f}')

def run_benchmark(ngrams: list, nsims: int) -> None:
    ts = np.linspace(1, 600, 200, dtype=int)
    js = jaccard_similarity(ngrams[0], ngrams[1])

    avg_times = np.zeros_like(ts, dtype=float)
    avg_error = np.zeros_like(ts, dtype=float)

    for i, t in enumerate(ts):
        error = np.zeros(nsims)
        times = np.zeros(nsims)

        for j in range(nsims):
            tic = time.perf_counter()
            ms = minhash_similarity(ngrams[0], ngrams[1], t)
            toc = time.perf_counter()
```

```
68              error[j] = abs(ms - js)
69              times[j] = toc - tic
70
71          avg_error[i] = np.mean(error)
72          avg_times[i] = np.mean(times)
73
74      np.savetxt('data/times.csv', np.column_stack((ts, avg_times)), delimiter=',')
75      np.savetxt('data/error.csv', np.column_stack((ts, avg_error)), delimiter=',')
76
77  def main() -> int:
78      parser = argparse.ArgumentParser(prog='fast-min-hash', description='implements a fully fledge minhash
            algorithm')
79
80      subparsers = parser.add_subparsers(dest='command')
81
82      similarity_parser = subparsers.add_parser('similarity', description='approximates document similarity')
83      similarity_parser.add_argument('-i', '--input', nargs=2, type=argparse.FileType('r'), required=True, help=
            'path to input files')
84      similarity_parser.add_argument('-n', '--ngram', type=int, required=True, help='ngram size')
85      similarity_parser.add_argument('-t', '--token', type=str, required=True, choices=['char', 'word'], help='
            token to use')
86
87      benchmark_parser = subparsers.add_parser('benchmark', description='runs a benchmark')
88      benchmark_parser.add_argument('-i', '--input', nargs=2, type=argparse.FileType('r'), required=True, help='
            path to input files')
89      benchmark_parser.add_argument('-n', '--ngram', type=int, required=True, help='ngram size')
90      benchmark_parser.add_argument('-t', '--token', type=str, required=True, choices=['char', 'word'], help='
            token to use')
91      benchmark_parser.add_argument('-s', '--nsims', type=int, required=True, help='number of times to repeat
            experiment')
92
93      args = parser.parse_args()
94      ngrams = list(
95          starmap(
96              build_ngrams,
97              zip(map(read_file, args.input), [args.ngram] * len(args.input), [args.token] * len(args.input))
98          )
99      )
100
101     match args.command:
102         case 'similarity':
103             run_similarity(ngrams)
104         case 'benchmark':
105             run_benchmark(ngrams, args.nsims)
106         case _:
107             parser.print_usage()
108
109     return 0
110
111 if __name__ == '__main__':
112     raise SystemExit(main())
```

Listing 3. fast-min-hash.py